

ISOS PROJECT

Introduction

The goal of this project is to understand how a dynamic loader works.

A dynamic loader loads shared libraries (.so files -> ELF format) at runtime, maps them into memory, and resolves symbols so that functions can be executed.

In this project, we re-implement this mechanism from scratch by building a custom loader called `isos_loader`. The objective is to load a shared library and execute its functions without relying on the system's dynamic loading facilities (`dlopen` and `dlsym`).

To achieve this, we implemented all the steps required for dynamic loading. In this report, we follow the logical progression of the project through the different challenges described in the guidelines. For each challenge (and bonus), we present the problem, the implementation choices, and the difficulties encountered.

Challenges

Challenge 1

For this challenge, we set up the environment, and we will rely for now on `dlopen` and `dlsym` just to see if works as expected before re-implementing them later in the project.

We also set up the CI pipeline by following the provided instructions and used the logs generated after each analysis (after each push) to debug potential issues.

For argument parsing, we used `argp`, which needed becoming familiar with its documentation. The required features (such as `--help`) were implemented, I added additional options like `--usage` and `--version`.

This challenge served as an initialization phase. Despite some adaptation time to understand the CI logs, no major difficulties were encountered.

Challenge 2 : Preliminary checks in the ehdr

In this challenge, we established the core foundations of the project.

We first set up the Makefile with the appropriate flags. Then, we created `elf_parser.h`, where we defined a structure called `dl_handle` to represent a dynamically loaded library. This structure is a

simplified equivalent of the handle returned by `dlopen` in the standard library. It contains a file descriptor (`fd`) to access the ELF file content, as well as a copy of its ELF header (`Elf64_Ehdr`) to store metadatas. This handle is designed to be extended with additional fields later in the project.

We then implemented the function `elf_open_and_check_ehdr`, which performs the required checks on the ELF header. These checks ensure that the file is a valid ELF binary, that it is 64-bit, dynamically linked, has the correct header size, and contains at least one program header.

These verifications are essential to prevent invalid or malformed binaries from being processed by the loader, which could lead to problems in later stages.

The main difficulty in this challenge was understanding the structure and meaning of the ELF header fields. This was solved by constantly having the ELF documentation while working and analyzing the role of each fields.

Challenge 3: Find the PT_LOAD segments

This challenge focuses on segments, and more specifically on PT_LOAD segments, which correspond to portions of the file that must be loaded into virtual memory during execution.

To achieve this, we iterated over the program headers to identify all segments of type PT_LOAD. Once identified, several checks were performed: 1 ensuring that there is at least one loadable segment, 2 verifying that the first PT_LOAD segment covers the program header table, 3 ensuring that segments are ordered by increasing virtual address (`p_vaddr`), 4 confirming that segments do not overlap.

These checks guarantee a valid and consistent memory. Preventing segment overlap avoids memory corruption, and enforcing ordering ensures that the memory mapping can be performed correctly.

These features were implemented in the function `elf_find_load_segments` in `elf_parser.c`.

One of the main difficulties was understanding how virtual addresses are organized and how segment overlap can occur. This issue was resolved by analyzing memory layouts and by adopting clearer variable naming conventions (like: `seg0_start` instead of directly accessing structure fields). More generally for this project we decided to use explicit and descriptive names for variables, functions, and structure fields in order to reduce misunderstandings and make debugging easier.

Challenge 4: The labyrinth of mmap

This challenge focuses on loading the segments into virtual memory while ensuring continuity and correct page alignment.

We implemented this logic in the function `seg_load_mem()` in `elf_parser.c`.

First, we reserve a contiguous virtual memory region large enough to contain all PT_LOAD segments, starting from address 0x00. The total size of this region was calculated in Challenge 3. Then, we compute the base address using the formula:

base address = starting address of a segment – `vaddr` of the same segment

Each segment is then mapped into memory using `mmap`, taking into account page alignment. Since segments may not start on page boundaries, adjustments are required by rounding addresses and offsets to page-aligned values.

Finally, appropriate memory permissions are applied to each segment.

The main difficulty in this challenge was understanding how mmap works, especially regarding page alignment and offset constraints. This was resolved by rounding operations to ensure that all mappings respect page boundaries.

Challenge 5: Dynamic relocations

This challenge is to perform dynamic relocations. Since the addresses used in the ELF file are virtual addresses relative to a base, they must be adjusted after loading the segments into memory.

The relocation information is stored in the PT_DYNAMIC segment, which contains, among other things, the relocation table (.rela.dyn). We iterate over each entry in this table and apply the required address adjustments.

A key issue encountered in this challenge was related to memory permissions. In Challenge 4, segments were mapped with their final permissions (e.g., read-only or executable). However, applying relocations requires write access to the affected memory regions. Therefore, it was necessary to temporarily modify the permissions to allow writing, perform the relocations, and then restore the original permissions.

Initially, we forgot to restore the original permissions after relocation, which resulted in all segments remaining writable. This issue was corrected by a reset of the proper permissions after the relocation phase.

Challenge 6: Call the exported symbols

In this challenge, we implement `my_dlsym`, whose role is to resolve a function name into its corresponding address in memory.

To achieve this, we created a custom symbol table that associates function names with their addresses (previously corrected during the relocation phase in Challenge 5). To make this table easily accessible, we placed it at a known location (the entry point) during compilation (`gcc -e`).

Using this table, `my_dlsym` can retrieve the address of a function based on its name, allowing the program to call it.

The difficulty encountered in this challenge was due to an incomplete implementation of relocations in Challenge 5. One type of relocation had been forgotten, which caused a bug when resolving symbols. After fixing the relocation handling, function calls worked as expected.

Bonus #2: Obfuscation

In this bonus, the goal is to hide the shared library inside the executable instead of storing it as a separate file on disk.

The library is encrypted using a simple XOR cipher (with key 42, the answer to the ultimate question of life, the universe, and everything), and appended to the executable. At runtime, the loader detects this embedded payload, decrypts it in memory, and loads it without ever writing it to disk.

To implement this, we created `encrypt.c`, which encrypts `libfoo.so` and appends its size (encoded on 8 bytes) at the end of the file. The Makefile automatically injects the encrypted payload into `isos_loader` after compilation.

At startup, the function `extract_embedded_payload()` reads these 8 bytes to locate the payload, decrypts it, and writes it into a file descriptor created with `memfd_create`, making it accessible via `/proc/self/fd/<n>`.

The main difficulty was managing the initialization order. The embedded payload had to be detected before parsing command-line arguments; otherwise, the argument parser could not determine whether the first argument was a library path or a function name. Additionally, handling the case where a library is explicitly provided via the command line required by introducing a flag (`lib_overridden`) to determine which library should be used.

CI warnings

During the CI analysis, some warnings were reported.

Most warnings concern the use of standard functions such as `fprintf`, `sprintf`, and `memset`, flagged as insecure because they do not use the C11 “_s” variants. In our context, these are not critical since all inputs are controlled.

A warning was also raised regarding the use of a potentially tainted value (`size`) for memory allocation in `extract_embedded_payload`. Although this value comes from the binary itself, it is generated at build time and not from external input. A bound check could improve robustness but was not implemented.

Another warning highlights a possible NULL dereference after `malloc`. This is a valid concern and could be fixed with an additional check.

Finally, a file descriptor leak was detected in `elf_open_and_check_ehdr`. An attempt was made to handle cleanup using a common error path, but the analyzer still reports a potential leak. The issue is understood.

Overall, these warnings do not affect the correctness of the loader in its intended use case but due to a lack of time we didn't correct the ones that are still happening.

Conclusion

This project provided a practical understanding of how a dynamic loader operates internally, from parsing ELF files to mapping memory and resolving symbols. Re-implementing these mechanisms helped a lot on understanding it.

Due to time constraints, Challenge 7 and Bonus 1 (which depends on Challenge 7) were not implemented. Additionally, some remaining CI warnings were identified but not fully resolved.